Mastering Market Peaks and Valleys: Advanced Trading with Deep Reinforcement Learning

Tran Hoang Long*
FPT University
Ho Chi Minh, VietNam
longthse182487@fpt.edu.vn

Nguyen Dinh Vu Hai*
FPT University
Ho Chi Minh, VietNam
haindvse182676@fpt.edu.vn

Nguyen Tran Phuong Anh*
Industrial University of Ho Chi Minh City
Ho Chi Minh, VietNam
23701771.anh@student.iuh.edu.vn

Nguyen Thanh Hoa*
FPT University
Ho Chi Minh, VietNam
hoantse183091@fpt.edu.vn

ABSTRACT

This research focuses on developing a sophisticated financial agent capable of autonomously learning from past transactions to make optimized trading decisions, ultimately aiming to maximize trading profits. The agent is designed to strategically buy or sell all stocks when market conditions reach perceived peaks or bottoms, or to hold when neither action is ideal. This dynamic decision-making process relies on advanced deep learning algorithms, including Proximal Policy Optimization (PPO), Advantage Actor-Critic (A2C), and Deep Q-Learning (DQN). These algorithms enable the agent to continually refine its performance, improving its trading strategies based on real-time market data and historical insights. Beyond simply automating the trading process, the agent strives to maximize returns even during volatile, high-risk periods in the financial market. We assessed the model's effectiveness using metrics such as Max Drawdown, Profit and Loss (PnL), and Return on Investment (ROI), achieving promising results of 61.936% for Max Drawdown, 7104.491 in PnL, and 71.044% in ROI, underscoring the agent's potential to navigate complex financial conditions while maximizing profitability.

KEYWORDS

Deep learning, Reinforcement learning, Q-Learning, Trading

ACM Reference Format:
Tran Hoang Long, Nguyen Dinh Vu Hai, Nguyen Tran Phuong Anh, and Nguyen Thanh Hoa. 2018. Mastering Market Peaks and Valleys: Advanced Trading with Deep Reinforcement Learning. In . ACM, New York, NY, USA, 7 pages. https://doi.org/XXXXXXX.XXXXXXX

*All authors contributed equally to this research.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, July 2017, Washington, DC, USA
© 2018 ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

In today's dynamic financial markets, investors face the ongoing challenge of making trades that maximize returns amid constant fluctuations. The sheer unpredictability of market trends and the overwhelming volume of data create difficulties in executing accurate and timely trades. Consequently, the development of a bot capable of automating trades with precision has become essential.

This paper presents a novel trading agent that utilizes reinforcement learning algorithms to learn from historical trades and execute automatic, data-driven trading decisions. Unlike conventional strategies based on static rules and indicators, our bot dynamically adapts to its market interactions, continually refining its trading approach in real time.

The agent's architecture incorporates advanced technologies, including PyTorch[10] and Stable Baselines3[12], frameworks tailored for deep learning and machine learning applications. These frameworks form a robust foundation for training and deploying the bot, ensuring both speed and reliability. The dynamic computation graph in PyTorch, combined with Stable Baselines3's suite of reinforcement learning algorithms, significantly enhances the agent's learning efficiency and precision.

By leveraging deep learning and cutting-edge technologies, the agent provides an advanced solution for investors looking to automate their trading workflows and enhance returns. It processes vast amounts of market data, identifying patterns and making data-driven decisions that adapt to changing market conditions. This automation minimizes the need for manual intervention and helps investors avoid emotional biases, allowing for more consistent and optimized trading outcomes.

2 RELATED WORKS

The paper "Deep Reinforcement Learning Robots for Algorithmic Trading: Considering Stock Market Conditions and U.S. Interest Rates" [9] inspired the development of a bot capable of autonomously analyzing and trading. It explored various reinforcement learning techniques, including deep-Q-learning, A2C[13], and PPO[5], which address dynamic decision-making challenges typical of financial markets. Emphasis was placed on the potential of these approaches to create intelligent systems that can learn from market data and make optimal trading decisions. Particularly insightful was the explanation of the Reinforce Learning (RL) training process,

covering elements such as observations, actions, rewards, and the integration of neural networks.

The paper also achieves significant performance improvements over traditional methods. Leveraging the PPO algorithm and a multi-stock strategy, it incorporates data from Apple Inc. stock, the S&P 500 index, and US interest rates to enhance accuracy and stability. The model's adaptability to market volatility and ability to learn from historical data are noted as key strengths. Despite these advancements, the need for further improvements is acknowledged, such as integrating value-based algorithms and utilizing larger datasets that include news, social media, and analyst reports. These enhancements could further refine the model's performance and decision-making capabilities.

3 METHODOLOGY

To explain our approach in detailed, we will first provided an overview of the trading context, describing the market conditions and relevant factors influencing our model. We will then outline the environment setup, including key parameters and constraints, and introduce the reward function that we made to guides the model's decision-making process. Finally, with those foundations being established, we will discuss our implementation of the Deep Q-learning architecture[8].

3.1 Trading context:

3.1.1 ***Total budget:*** For this model, we will use the **META's stocks** data from **01/01/2018 - 01/01/2022** as the train data and **01/01/2022 - 01/01/2024** as the test data. We have allocated a total budget of **10,000$**. This amount represents the maximum capital that will be used throughout the training and decision-making process. By setting a fixed budget, we aim to control the level of risk exposure while allowing the model to explore different strategies within a predefined financial limit. This approach not only helps manage potential losses but also ensures that the model operates under realistic financial constraints, similar to what would be encountered in a real-world scenario. Using this **10,000$** budget, the model will make decisions with the goal of maximizing returns while carefully managing the allocation of funds to minimize unnecessary risks.

3.1.2 ***Order size:*** To maximize the benefits of market growth, we will implement a strategy that involves setting the order size to sell the entire stock position when it reaches its peak price and then using the full budget to repurchase the stock when it drops to its lowest point. This method is designed to ensure that we capitalize fully on the stock's upward movement by securing profits at the optimal time, just before any potential downturn or price correction. By selling at the peak, we lock in the highest possible returns, preventing losses that could result from holding onto the stock as its value declines. Similarly, reinvesting the entire budget when the stock hits its bottom, it allows us to maximize the number of shares acquired at the most favorable price, setting the stage for significant gains when the market trends upward again. This approach aims to create a cycle of buying low and selling high, amplifying the overall growth potential of our investment portfolio while strategically managing risk. By aligning our trading decisions with these principles, we aim to fully exploit market fluctuations to achieve sustained and substantial profit gains over time [6].

3.2 Create an environment for Reinforcement learning

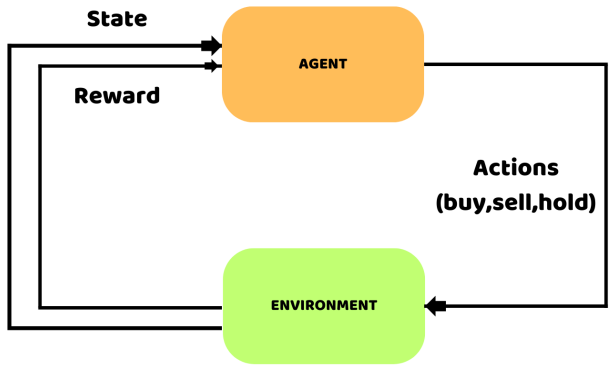

Figure 1: Agent interaction with the environment

The figure 1 will show how our agent interacts with an environment by taking actions in trading. Based on the current state (such as stock prices), the agent takes an action, which then affects the environment. In response, the environment provides a reward based on the success of that action, and updates the state. The agent uses this feedback loop to learn and improve its future decisions over time.

3.2.1 ***Total action:*** In this model, there are a total of three possible actions that the agent can take, which are numbered as [0, 1, 2], corresponding to [buy, hold, sell]. These actions represent the fundamental decisions available to the agent. Specifically, "Buy" instructs the agent to purchase assets, "Sell" signals it to offload assets, and "Hold" directs it to maintain its current position[14]. These three actions form the core of the agent's decision-making process and will directly influence its learned policy as it interacts with the environment and adjusts its strategy based on the outcomes of each decision. The agent learns to optimize its actions by balancing between these choices to maximize profits or minimize losses as it encounters various market conditions .

3.2.2 ***Return State Steps:*** In our approach, we will begin by setting the reward value to 0 for the initial step. Afterward, the model will retrieve the 5 closest price values from the starting date and calculated the percentage change for each consecutive day that has passed. This calculation is based on the formula:

$$\text{Percentage change} = \frac{Price_{i+1} - Price_i}{Price_i} \times 100$$

This formula measures the relative change in stock price from one day to the next, expressed as a percentage. By applying it to the nearest five price values, we can track how the stock's price fluctuates over time, reflecting both increases and decreases in market value. The daily percentage change metric is particularly relevant for analyzing market behaviors over specific periods, helping

in identifying patterns such as seasonal effects or cyclical movements[1]. The percentage change will be adjusted accordingly for each day to provide a clearer picture of daily price dynamics, allowing the model to capture trends and generate more accurate predictions for future actions. Through this method, we aim to understand price behavior over a series of consecutive days, which will ultimately influence the decision-making process regarding when to buy or sell stock based on observed changes in the market conditions.

The sequence of actions in this model will continue until the agent reaches the final value point, at which the decision-making process will cease. This means the agent will keep executing actions—buying, holding, or selling—based on its learned policy, adjusting to market changes, and optimizing for the best possible outcome. Once the agent reaches this predetermined endpoint, where the final value of the asset is determined or the agent's goal is met, no further actions will be taken. This final value point acts as a natural stopping criterion for the trading process, signaling that the model has completed its task of navigating the market and executing trades to achieve the desired result.

3.3 Reward Function

The reward function is a fundamental component of reinforcement learning, serving as a guide for agents to evaluate their actions within an environment and learn optimal behaviors over time. It provides feedback on the consequences of an agent's actions by assigning scalar values, which can be positive or negative, thereby influencing the agent's future decision-making processes[2]. The design of the reward function directly impacts the efficiency and effectiveness of the learning process, as it shapes the agent's understanding of its goals and the strategies required to achieve them[3]. Because we are trying to make the model to specialized in trading we chose to set the reward for 3 actions [buy, hold, sell]

3.3.1 ***Reward for Buy:*** To determine the reward for the "Buy" action, we calculated it by setting the selling price as the closed price from the initial state, even if no stocks have been sold yet. The reward for the buy action is computed using the following formula:

- **If holding money (stock=0):**

$$reward_{buy} = \text{latest sell price} - \text{current price}$$

- **Otherwise:**

$$reward_{buy} = 0$$

This formula helps the model determine the optimal time to buy. For instance, if the price increases after the initial state, the reward will be negative, signaling to the model that buying at this moment would have been a poor decision, as it would have been better to buy earlier. As prices increase, the reward becomes more negative, reinforcing this lesson for the model. On the other hand, if the price decreases, the reward becomes larger, indicating that it was a good time to buy, and this positive feedback helps improve the model's learning. This reward structure encourages the model to seek out buying opportunities when prices are low, which aligns with the goal of maximizing long-term profitability.

3.3.2 ***Reward for Sell:*** To contrast the reward for the "Buy" action, the "Sell" action which we will calculate by comparing the current price with the last buy price. The reward for the sell action is given by the following formula:

- **If holding stock(money=0):**

$$reward_{sell} = \text{current price} - \text{latest buy price}$$

- **Otherwise:**

$$reward_{sell} = 0$$

This formula is designed to guide the model in determining the optimal time to sell. If the price has risen since the last purchase, the reward will be positive, signaling to the model that selling at this point would generate a profit. The higher the price goes, the larger the reward, reinforcing the idea that selling after a price increase is a beneficial action. Conversely, if the price drops after buying, the reward will be negative, indicating that selling at this moment would lead to a loss. This negative feedback helps the model learn to avoid selling when prices are low, encouraging it to wait for more favorable market conditions. The reward system thus promotes the model's understanding that selling at a higher price yields better outcomes and aligns with the goal of maximizing profit.

3.3.3 ***Reward for Hold:*** The reward for the "Hold" action will utilize both the "Buy" reward function (denoted as (1)) and the "Sell" reward function (denoted as (2)), but with a slight modification. After a buy action has occurred, we will apply the sell reward function (2), and after a sell action has occurred, we will apply the buy reward function (1). The idea is to determine whether the agent should hold its current position or proceed with buying or selling. To achieve this, the "hold" reward will be reduced in comparison to the rewards for "buy" and "sell" actions. This is done by multiplying both reward functions by an ****Influence Reduction Parameter(IRP)**** which reduced the impact of the "Hold" action to the model compared to the "Buy" and "Sell", set at **0.1** in this model, illustrated by the formula:

- **If holding money (stock=0):**

$$reward_{hold} = reward_{buy} \times IRP$$

- **Else if holding stock (money=0):**

$$reward_{hold} = reward_{sell} \times IRP$$

To minimize the impact of the hold reward and make it less influential than the actual buying or selling actions. The reward for holding will remain reduced until the rewards from both functions—buying and selling—reach a point where maximum profits can be realized. In other words, this strategy encourages the agent to hold its position until it can either sell at the peak price or buy at the lowest price during the period (a strategy often referred to as "bottom fishing"). When the "buy" or "sell" rewards reach their respective optimal points, they will be 10 times greater than the hold reward, signaling that it is time to take action rather than continue holding. This design ensures that holding is only used as a temporary strategy and that the agent is rewarded more for making timely and profitable decisions to buy or sell.

3.4 Design the neural network of Deep Q-learning

Since we're dealing with time series data, traditional Deep Q-learning using a multilayer perceptron (MLP)[11] isn't effective because it

cannot capture temporal dependencies essential for making sequential decisions. In Q-learning, the agent needs to learn from past actions and outcomes to make optimal future decisions. To handle this, we integrate LSTM (Long Short-Term Memory) networks[4], which are specifically designed to retain information over time. LSTMs can remember important details from previous time steps and use that information to influence current decisions, making them ideal for tasks like stock trading. By adding LSTM in Deep Q-learning, we ensure that the model can effectively learn from time series data and make better predictions in a dynamic environment.

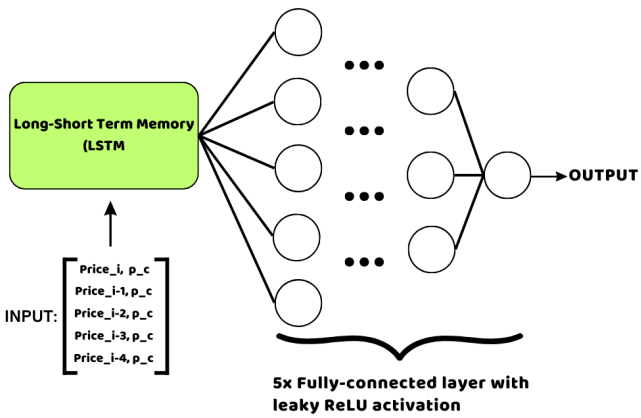

Figure 2: The structure of the deep Q-learning

The model structure presented in the Figure 2 consists of two main components: an LSTM (Long Short-Term Memory) layer and a sequence of fully-connected layers with leaky ReLU[7] activation. The input to the model is the Return State Steps of the environment. These input values, representing stock prices at different points in time, are fed into the LSTM layer. LSTM is particularly well-suited for handling time series data as it can capture long-term dependencies, retaining relevant information from the past while forgetting irrelevant data through its memory cell architecture. This allows the model to learn the temporal dynamics and trends of stock prices.

Once the data has passed through the LSTM, the output is then fed into a series of fully connected layers, which use leaky ReLU activation functions. The use of leaky ReLU prevents neurons from "dying" by allowing a small, non-zero gradient when the input is negative, improving the learning capability of the model. These layers form the "Deep" part of Deep Q-learning, transforming the processed time-series data into a more complex, abstract representation. The final output is the Q-values corresponding to the possible actions (buy, hold, sell). These Q-values guide the agent in determining the best action to take at each step based on the expected future rewards. By combining LSTM to capture time-dependent features and fully connected layers for deeper representations, the model effectively learns an optimal policy for trading decisions in a reinforcement learning setting.

4 TRAINING

4.1 Training configuration

The system configuration is used to train the Deep Q-Learning algorithm with a total training time of 2 hours, 10,000 total time steps, using an RTX 3060 GPU.

4.2 Deep Q-learning hyperparameters

These settings define the hyperparameters used for training the model:

- **Learning rate**: 0.0001
- **Buffer size**: 10^5
- **Gamma (discount factor)**: 0.99
- **Train frequency**: Every 4 steps

5 EVALUATION METRICS

The following metrics evaluate investment performance and risk management, providing insights into different aspects of an investment strategy: **Max Drawdown**, **ROI (Return on Investment)**, and **PnL (Profit and Loss)**. Each metric serves a specific purpose, allowing investors to better understand and optimize their portfolios.

5.1 Max Drawdown

Max Drawdown measures the maximum decline from a peak to a trough in the value of a portfolio or asset within a specified period. This metric represents the greatest potential loss an investor could experience during a period of market decline. Max Drawdown is particularly useful in risk management, as it enables investors to gauge the level of vulnerability in their portfolios and assess recovery potential following a downturn.

Max Drawdown Formula:

$$\text{Max Drawdown} = \frac{\text{Trough Value} - \text{Peak Value}}{\text{Peak Value}} \tag{1}$$

Where:

- **Peak Value:** denotes the highest value reached by the asset or portfolio before the decline.
- **Trough Value:** signifies the lowest point in the asset's or portfolio's value during the decline.

Significance. Max Drawdown is crucial for risk assessment, providing investors with a clear understanding of maximum potential losses. This insight enables them to adjust their investment strategies to safeguard capital in times of market volatility.

5.2 ROI (Return on Investment)

ROI is a performance measure used to evaluate the efficiency or profitability of an investment relative to its initial cost. Expressed as a percentage, ROI allows investors to compare the profitability of different investments and make well-informed decisions based on the return yielded for the amount invested.

ROI Formula:

$$\text{ROI} = \frac{\text{Net Profit}}{\text{Investment Cost}} \times 100 \tag{2}$$

Where:

- **Net Profit:** represents the total return or profit generated after deducting all related costs.
- **Investment Cost:** refers to the initial expenditure associated with the investment.

Significance. ROI enables investors to assess the effectiveness of their investments by illustrating the return relative to costs. This metric aids in determining whether an investment has been profitable and if it is worth maintaining or expanding.

5.3 PnL (Profit and Loss)

PnL reflects the financial outcome of an investment by calculating the difference between total revenue and total expenses. This metric can indicate a positive outcome (profit) or a negative outcome (loss), providing an overview of an investment's financial performance over a designated timeframe.

PnL Formula:

$$\text{PnL} = \text{Revenue} - \text{Expenses} \tag{3}$$

Where:

- **Revenue:** is the total income generated from the investment or business operation.
- **Expenses:** encompass all costs or expenditures related to the investment or operation.

Significance. PnL offers investors a comprehensive view of the financial health of an investment, allowing them to evaluate whether it meets profit expectations. With this metric, investors can modify strategies to maximize gains or mitigate losses.

5.4 Summary

- **Max Drawdown** is essential for assessing risk by identifying potential maximum losses.
- **ROI** measures the profitability of an investment relative to its cost, facilitating comparative analysis across different investments.
- **PnL** provides an overarching view of financial performance, highlighting overall gains or losses.

Together, these metrics deliver a comprehensive evaluation of investment performance, helping investors make informed decisions that align with their financial goals.

6 RESULTS

Table 1: Comparison of Metrics for Different Algorithms

Algorithm	Max Drawdown	PnL	ROI
Deep Q-Learning	61.937	7104.492	71.045
A2C	21.686	7741.155	77.412
PPO	61.969	5596.655	55.967

The comparison table presents the performance of three reinforcement learning algorithms: Deep Q-Learning, A2C, and PPO.

Deep Q-Learning demonstrates superior performance compared to PPO in several aspects. With a Max Drawdown of 61.937%, it is higher than A2C (21.686%) but slightly lower than PPO (61.969%), indicating relatively good risk management capabilities. In terms of Profit and Loss (PnL), Deep Q-Learning achieves 7104.492, significantly higher than PPO's 5596.655, showcasing its ability to generate better returns.

The Return on Investment (ROI) for Deep Q-Learning is 71.045%, which is higher than PPO's 55.967%, although still lower than A2C's 77.412%. This suggests that while A2C excels in overall performance, Deep Q-Learning maintains a competitive edge over PPO and demonstrates a strong ability to generate profits.

In summary, Deep Q-Learning outperforms PPO in key performance metrics and has some limitations compared to A2C, but it remains a notable choice among the three algorithms analyzed.

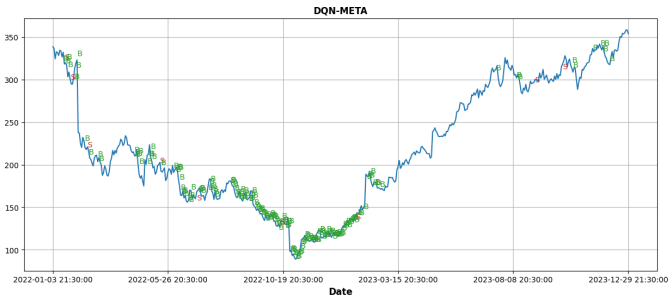

Figure 3: Orders placed by Deep-Q Learning algorithm

In October 2022, the algorithm executed several buy orders at the market bottom (100–150 USD), demonstrating its ability to identify favorable opportunities to accumulate shares at low prices. This reflects its capability to learn from environmental signals and capitalize on market downturns. However, according to the predefined strategy in the "Trading Context," the first buy order effectively sold off the entire stock position. Therefore, the subsequent buy orders until the next sell signal did not involve purchasing additional shares but were equivalent to hold orders, aiming to maximize returns without increasing exposure. By March 2023, as META's stock price recovered, the algorithm opted to maintain its existing positions rather than engaging in further trades, demonstrating an optimized strategy for maximizing returns by holding during a bullish market trend without overtrading. This indicates the algorithm's ability to recognize the upward trend and avoid unnecessary trades, thus optimizing long-term profits. In September 2023, multiple sell orders were executed near the price peak (350 USD), reflecting the algorithm's capacity to identify market tops and secure profits before potential reversals. As with the earlier buy orders, the first sell order liquidated the entire stock position, and subsequent sell orders acted as placeholders, equivalent to holding a zero position to lock in gains. Overall, the algorithm demonstrates a balanced trading strategy, characterized by buying at low prices, holding positions in favorable market conditions, and selling near market peaks. This highlights its strong adaptive decision-making and effective market analysis.

7 DISCUSSION

7.1 Overview

This study focuses on developing a financial trading bot that utilizes the Deep Q-Learning algorithm to optimize trading profits. The bot autonomously learns from previous transactions and adjusts its trading strategies based on prior decisions. To evaluate its effectiveness, the research also compares the performance of two other algorithms: Proximal Policy Optimization (PPO) and Advantage Actor-Critic (A2C).

7.2 Benefits

- **Automated Learning**: Trading bots learn from past transactions, continuously improving their strategies without human intervention. This leads to more informed decisions based on empirical data rather than intuition.
- **Dynamic Adaptation**: By leveraging RL, bots can adapt to changing market conditions, identifying patterns and trends that may not be obvious to human traders.
- **Profit Optimization**: Bots aim to maximize profits while minimizing risk, applying strategies such as buying low and selling high based on learned policies.

7.3 Drawbacks

- **Data Dependence**: The effectiveness of trading bots heavily relies on the quality and quantity of historical data. Inaccurate or incomplete data can lead to poor performance.
- **Risk of Overfitting**: There is a danger of models overfitting to historical data, which can result in poor generalization to future market conditions.
- **Market Volatility**: Financial markets are inherently unpredictable; sudden market changes can disrupt learned strategies, leading to significant losses.

7.4 Challenges

- **Designing Reward Functions**: Creating an effective reward function is crucial. If not designed properly, it can misguide the learning process, resulting in suboptimal trading strategies.
- **Hyperparameter Tuning**: Finding the right combination of hyperparameters for algorithms like DQN, PPO, and A2C are essential for optimal performance and can be time-consuming.
- **Risk Management**: Developing robust risk management strategies to mitigate potential losses while maximizing profits remains a significant hurdle in automated trading.
- **Computational Demands**: Training complex RL models can be resource-intensive, requiring powerful hardware and substantial time investment.
- **Market Regulations**: Compliance with financial regulations and trading rules is crucial, and navigating these can complicate the implementation of trading bots.

7.5 Improvements

To enhance the performance of trading bots, several approaches can be applied:

- **Reward Function Optimization**: Experimenting with different reward functions ensure that the bot can learn the most effective strategies.
- **Combining Multisource Data**: Utilizing data from various sources, such as economic indicators and market sentiment, provides a broader perspective and improves trading decisions.
- **Sensitivity Analysis**: Conducting sensitivity analyses helps to better understand how different factors influence the bot's decisions, allowing for appropriate model adjustments.
- **Improving the Training Environment**: A crucial improvement involves refining the training environment to minimize confusion when the bot places buy or sell orders without owning stocks or when the account balance is zero. In such cases, the bot should recognize that no action can be taken and instead classify these situations as hold orders, where no transaction occurs. This adjustment ensures the bot avoids invalid trades and better mirror realistic market conditions, thereby improving the learning process and overall performance.
- **Penalty Mechanism for Misplaced Orders**: Implementing a penalty mechanism for incorrect trades helps the bot learn from its mistakes. When the bot incorrectly places a buy, sell, or hold order that results in a loss, it should be penalized proportionally to the extent of that loss. For example, buying without sufficient balance or selling without owning the stock would trigger penalties. This system reinforces correct decision-making, encouraging the bot to avoid costly errors in the future. While introducing penalties is crucial, determining an appropriate threshold for penalties is still being evaluated to ensure the balance between effective learning and deterrence.

8 CONCLUSION

This research presents a comprehensive analysis of reinforcement learning algorithms, specifically Deep Q-Learning, A2C, and PPO, within the context of automated financial trading. The study confirms that each algorithm exhibits unique strengths and limitations when applied to the dynamic and unpredictable environment of the financial markets.

The trading environment simulates market behavior, where an RL agent learns how to trade by interacting with this environment. It uses observations (e.g., prices, indicators) to make decisions (buy/sell/hold), and it optimizes those decisions based on rewards tied to profits or portfolio performance

The empirical results demonstrate that Deep Q-Learning offers a balanced performance, excelling in Profit and Loss (PnL) metrics and achieving a competitive Return on Investment (ROI), making it a viable choice for scenarios that require efficient profit generation with moderate risk management. A2C, on the other hand, shows superior ROI, suggesting that it is well-suited for maximizing returns, particularly in environments where long-term gains are prioritized over short-term fluctuations. PPO, while not leading in any specific metric, offers stable performance across various market conditions, showcasing its robustness and adaptability to different trading scenarios.

The research underscores the potential of reinforcement learning algorithms to adaptively learn from market trends, effectively manage risks, and make informed decisions based on evolving data. The success of Deep Q-Learning in recognizing market downturns for accumulation, maintaining positions during bullish trends, and timing exits near market peaks highlights the algorithm's capacity to navigate complex trading environments. Meanwhile, A2C and PPO demonstrate their respective strengths in optimizing returns and maintaining consistent performance under diverse market conditions.

The findings of this study contribute to the ongoing development of intelligent trading systems and adaptive financial technologies.

REFERENCES

[1] Chiaku Chukwuogor. [n. d.] Stock market returns analysis, day-of-the-week effect, volatility of returns: evidence from european financial markets 1997-2004.

[2] D Ernst and A Louette. 2024. Introduction to reinforcement learning, (Jan. 2024). https://damien-ernst.be/wp-content/uploads/2024/03/f90de-introduction_to_reinforcement_learning.pdf.

[3] J Eschmann. 2021. Reward function design in reinforcement learning, (Jan. 2021). https://link.springer.com/chapter/10.1007/978-3-030-41188-6_3.

[4] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural computation*, 9, (Dec. 1997), 1735–80. DOI: 10.1162/neco.1997.9.8.1735.

[5] Filip Wolski John Schulman. 2017. Proximal policy optimization algorithms, (Jan. 2017). https://arxiv.org/abs/1707.06347.

[6] HE Leland. 1974. Production theory and the stock market, (Jan. 1974). https://www.jstor.org/stable/3003095.

[7] Guifang Lin and Wei Shen. 2018. Research on convolutional neural network based on improved relu piecewise activation function. *Procedia computer science*, 131, 977–984.

[8] Volodymyr Mnih et al. 2015. Human-level control through deep reinforcement learning. *nature*, 518, 7540, 529–533.

[9] Ji-Heon Park, Jae-Hwan Kim, and Jun-Ho Huh. 2024. Deep reinforcement learning robots for algorithmic trading: considering stock market conditions and u.s. interest rates. *IEEE Access*, PP, (Jan. 2024), 1–1. DOI: 10.1109/ACCESS.2024.3361035.

[10] Adam Paszke et al. 2019. Pytorch: an imperative style, high-performance deep learning library. (2019). https://arxiv.org/abs/1912.01703 arXiv: 1912.01703 [cs.LG].

[11] Marius-Constantin Popescu, Valentina Balas, Liliana Perescu-Popescu, and Nikos Mastorakis. 2009. Multilayer perceptron and neural networks. *WSEAS Transactions on Circuits and Systems*, 8, (July 2009).

[12] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. 2021. Stable-baselines3: reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22, 268, 1–8. http://jmlr.org/papers/v22/20-1364.html.

[13] Adrià Puigdomènech Badia Volodymyr Mnih. 2016. Asynchronous methods for deep reinforcement learning, (Jan. 2016). https://arxiv.org/abs/1602.01783.

[14] Z Zhang, S Zohren, and S Roberts. 1911. Deep reinforcement learning for trading. *arXiv preprint arXiv:1911.10107*, (Jan. 1911). https://arxiv.org/abs/1911.10107.

Received ; revised ; accepted